Mitos em 30 segundos — Automação de YouTube Shorts

Guia completo: como funciona, como publicar 1 vídeo por dia sozinho, e como usar o Manus gastando o mínimo de tokens.

---

1. Resposta curta

Sim, dá para automatizar. E você não precisa pedir vídeo por vídeo ao Manus.

Todos os roteiros seguem a estrutura editorial fixa:

> **GANCHO → HISTÓRIA → SURPRESA → FINAL → CTA**

O gancho abre com uma curiosidade popular, como: “Você sabia que Zeus não era originalmente o deus mais poderoso?”. A história explica o contexto para quem nunca ouviu falar do mito; a surpresa entrega o fato pouco conhecido; e o final fecha com uma consequência ou imagem marcante. A pauta prioriza personagens conhecidos como Zeus, Medusa, Hades, Afrodite, Thor, Loki, Odin, Hércules, Aquiles, Anúbis, Iara e Saci.

A regra que resolve 90% do problema de custo:

> O vídeo diário **não** deve ser feito numa conversa com o agente. Ele deve ser feito por um **script Python determinístico** que roda sozinho. O agente Manus entra só para montar, corrigir e evoluir esse script — ou seja, algumas vezes por mês, não 30 vezes por mês.

Se você pedir “Manus, faça o vídeo de hoje” todos os dias, você paga **créditos de agente** (o modelo raciocina, lê arquivos, executa ferramentas) todo dia. Com o pipeline abaixo, o trabalho diário roda em Python e o custo por vídeo cai para **cerca de US$ 0,002** (roteiro) + **R$ 0** (voz, imagens, trilha, render).

Este projeto já está construído e funcionando em `/home/ubuntu/mythos-shorts`.

---

2. O que o pipeline faz em cada vídeo

```
1. Escolhe um tema                topics.py  (~45 temas, sem repetir)
2. Escreve o roteiro de 30s       1 chamada de LLM barato → JSON estrito
3. Narra em pt-BR                 edge-tts (voz neural gratuita)
   └─ e captura o tempo de CADA palavra
4. Busca arte de domínio público Wikimedia Commons (gravuras de Doré,
   vasos gregos, esculturas, iluminuras) — com cache permanente
5. Monta a trilha épica e luminosa  pasta assets/music/ ou composição pr
6. Renderiza com FFmpeg           1080x1920, Ken Burns, grade cinematográf
   legendas karaoke palavra por palavra, música com ducking
7. Publica no YouTube             API oficial, título/descrição/tags/hasht
```

Resultado: um Short vertical de ~30 segundos, com legenda sincronizada, arte de museu e trilha — feito sem nenhum custo de geração de vídeo, imagem ou voz.

Por que essa escolha de mídia

Elemento	Escolha	Motivo
Voz	edge-tts (`pt-BR-AntonioNeural` , pitch −8 Hz)	voz neural de qualidade, **grátis e ilimitada**, e entrega o timestamp de cada palavra (essencial para a legenda karaoke)
Imagem	Wikimedia Commons (domínio público)	gravuras e esculturas antigas combinam com o tema épico, **custo zero** e uso legal liberado
Trilha	compositor procedural em numpy + biblioteca local	nunca falta trilha, nunca repete, **custo zero**
Render	FFmpeg local	determinístico, sem limite de uso
Roteiro	LLM barato (gpt-5-mini, `reasoning=low`)	qualidade boa, **~US$ 0,002 por vídeo**

Se você quiser subir o nível depois, gere algumas imagens e trilhas com IA **uma única vez**, salve em `assets/images/` e `assets/music/` , e o pipeline passa a usá-las para sempre. Você paga uma vez, não todo dia.

---

3. Como usar (comandos)

```
cd /home/ubuntu/mythos-shorts

# teste completo SEM gastar LLM (usa roteiro fixo) — sempre use antes de r
python3 run_daily.py --dry-run --topic medusa

# gera o Short do dia (não publica)
python3 run_daily.py

# gera E publica
python3 run_daily.py --upload

# forçar um tema específico (veja os ids em topics.py)
python3 run_daily.py --topic curupira --upload

# produzir a semana inteira de uma vez, agendando cada vídeo num horário
python3 batch.py --count 5 --hour 20 --upload

# publicar como não listado / privado (para revisar antes)
python3 run_daily.py --upload --privacy unlisted
```

Cada execução grava um log em `logs/`, o MP4 em `output/` e um `.json` ao lado com título, descrição e tags — útil se você quiser subir na mão.

---

4. Configuração inicial (o único passo manual)

4.0 Pré-requisito

Você precisa de um canal do YouTube já criado na conta Google que vai publicar.

4.1 Ativar a API

1. Acesse https://console.cloud.google.com e crie um projeto (ex.: “Mythos Sombrio”).
2. **APIs e serviços** → **Biblioteca** → procure **“YouTube Data API v3”** → **Ativar**.

4.2 Tela de consentimento

1. **APIs e serviços → Tela de permissão OAuth**:
   - Tipo de usuário: **Externo**
   - Preencha nome do app, e-mail de suporte e e-mail do desenvolvedor.
2. Em **Público-alvo**, clique em **Publicar app** (mude de "Teste" para "Em produção").

> **Isto é importante:** em modo "Teste", o Google expira o refresh token em ~7 dias e sua automação morre toda semana. Publicando, o token dura indefinidamente (o app fica "não verificado", mas funciona normalmente para uso pessoal).

4.3 Criar a credencial

1. **Credenciais → Criar credenciais → ID do cliente OAuth**
   - Tipo de aplicativo: **App para computador**
2. Baixe o JSON e salve exatamente como:

```
/home/ubuntu/mythos-shorts/credentials/client_secret.json
```

4.4 Autorizar (uma única vez)

```
cd /home/ubuntu/mythos-shorts
python3 auth_setup.py
```

O script funciona **sem navegador no servidor**:

1. ele imprime um link — abra no **seu** navegador e autorize com a conta do canal;
2. o navegador vai tentar abrir `http://localhost:8080/?code=...` e mostrar **erro de conexão** — isso é esperado;
3. copie a **URL inteira** da barra de endereços e cole no terminal;
4. pronto: `credentials/token.json` é criado e nunca mais precisa repetir.

Confirme com:

```
python3 upload.py     # imprime o nome e o ID do canal autorizado
```

---

5. Como automatizar o "todo dia"

O pipeline está pronto (`post_now.sh` é o ponto de entrada). Falta só escolher **quem dispara** todo dia. Três caminhos, do mais simples ao mais independente:

Opção A — Manus ScheduleTask (comece por aqui)

Uma tarefa agendada que roda **um único comando**.

- **Ação a executar:** `bash /home/ubuntu/mythos-shorts/post_now.sh`
- **Frequência:** todo dia, no horário que você escolher (ex.: 09:00 de Brasília)
- **Custo:** uma execução curta por dia — mínimo possível, porque o agente só dispara o script e devolve o link.

Opção B — Um Cloud Computer (a partir de ~US$ 10/mês)

Uma máquina Linux persistente com `cron` de verdade. A automação roda **sem consumir nenhum crédito do Manus** e sem você abrir nada.

```
# exemplo de cron: todo dia às 12:00 UTC = 09:00 de Brasília
0 12 * * * cd /home/ubuntu/mythos-shorts && bash post_now.sh >> logs/cron
```

Opção C — Seu próprio computador (grátis)

Rode localmente com Agendador de Tarefas (Windows) ou `cron` (Linux/macOS). Custo zero, mas a máquina precisa estar ligada no horário.

O truque que reduz o custo de automação em 7x

Todos os caminhos ficam muito mais baratos com **produção em lote**:

```
python3 batch.py --count 5 --hour 20 --upload
```

Uma única execução gera 5 vídeos e agenda cada um para um dia e horário diferentes. Você troca “acordar a automação 7 vezes por semana” por **uma vez por semana**.

> **Atenção à cota do YouTube:** cada upload custa 1.600 unidades e a cota padrão é 10.000/dia — ou seja, no máximo ~6 uploads por dia. Para lotes maiores, peça aumento de cota no Google Cloud.

---

6. Como usar o Manus gastando o mínimo de tokens

A regra central

Nunca peça o conteúdo diário em conversa. Peça *infraestrutura*, não *produto*.

Você quer	Jeito caro	Jeito barato (use este)
30 vídeos	30 conversas, 30× roteiro+imagem+voz+render pelo agente	1 script que roda sozinho 30×
Roteiro do dia	“Manus, escreva o roteiro” (créditos de agente)	`script_gen.py` chama a LLM por US$ 0,002
Imagem do dia	gerar imagem com IA todo dia	Wikimedia Commons (grátis) + biblioteca que você acumula
Trilha do dia	gerar música com IA todo dia	compositor procedural (grátis)
Testar uma mudança	gerar um vídeo real	`python3 run_daily.py --dry-run`

Hábitos que economizam muito

1. **Mantenha o `agents.md`.** Toda sessão nova do Manus começa lendo `mythos-shorts/agents.md`, que descreve a arquitetura, os comandos e as regras. Sem ele, o agente redescobre tudo — e você paga essa redescoberta todo vez. **Atualize esse arquivo sempre que mudar algo estrutural.**
2. **Uma tarefa por objetivo.** “Ajuste a legenda” numa tarefa, “adicione 20 temas” noutra. Conversas longas reprocessam contexto a cada mensagem.

3. **Diga o arquivo e o comportamento**, não a teoria: *"em `render.py`, aumente a fonte da legenda para 84 e deixe o contorno 8"*. Pedidos vagos geram exploração caríssima.
4. **Peça o lote, não a unidade.** "Gere 5 vídeos" custa quase o mesmo que "gere 1" em overhead de agente, porque o trabalho pesado é do script.
5. **Use `--dry-run` para validar.** Testa todo o pipeline com roteiro fixo, gastando zero de LLM.
6. **Exija o log.** `logs/AAAAAMMDD.log` tem o passo a passo. Anexar o log numa tarefa de correção evita que o agente investigue do zero.
7. **Deixe os agentes fazerem trabalho paralelo.** Para tarefas independentes (ex.: revisar 20 roteiros), uma única tarefa com subagentes custa menos que 20 conversas.

Onde vale a pena gastar (pouco) com geração de IA

- **Uma sessão única de 10–20 imagens** no seu estilo visual, salvas em `assets/images/` → reaproveitadas para sempre.
- **2–3 trilhas** de estúdio em `assets/music/` → o pipeline usa automaticamente.
- **Uma thumbnail/identidade visual** para o canal.

Tudo isso é pagamento único, não recorrente.

---

7. Estratégia de canal (para o vídeo ser assistido)

O pipeline resolve a produção. Estes pontos resolvem o alcance:

- **Os 2 primeiros segundos decidem tudo.** O roteiro já é gerado com um *hook* de tensão; confira sempre e reescreva os fracos em `topics.py` (o campo `angulo` guia o roteiro).
- **Legenda karaoke é obrigatória** em Shorts: a maioria assiste sem som. Já está embutida, palavra por palavra.
- **Duração de 30 s** é ótima para retenção: dá para assistir 2 vezes e a taxa de conclusão sobe.
- **Consistência > perfeição.** 1 vídeo por dia no mesmo horário ensina o algoritmo e cria hábito.
- **Nicho fechado:** folclore brasileiro + mitologia comparada é um nicho com demanda alta e pouca concorrência com qualidade. Não misture com outros assuntos no mesmo canal.
- **Use `topics.py` como pauta editorial.** Cada tema tem um recorte sombrio já definido — é ali que você ajusta a linha do canal.

- **Evite conteúdo enganoso no título.** O YouTube penaliza clickbait que o vídeo não entrega; o gerador já é instruído a manter fidelidade ao mito.

---

8. Custos reais

Item	Custo
Roteiro (LLM)	~US$0,002 por vídeo ≈ **US$ 0,06/mês**
Voz	R$ 0
Imagens	R$ 0
Trilha	R$ 0
Render	CPU do ambiente
Automação (Opção A — ScheduleTask)	consumo mínimo diário do Manus
Automação (Opção B — Cloud Computer)	a partir de ~US$ 10/mês

Comparação: um editor freelancer cobrando R$ 30 por Short custaria **R$ 900/mês**. Aqui o custo é próximo de zero.

---

9. Arquivos do projeto

Arquivo	Papel
`config.py`	**toda** a configuração: voz, duração, canal, volume, loudness
`topics.py`	banco de temas (edite aqui para mudar a pauta do canal)
`script_gen.py`	geração do roteiro (1 chamada de LLM por vídeo)
`tts.py`	narração gratuita + timestamps por palavra
`visuals.py`	busca/cache de arte de domínio público
`music.py`	trilha (biblioteca local ou composição procedural)
`render.py`	renderização FFmpeg, legendas karaoke, mixagem
`upload.py`	publicação e agendamento no YouTube
`auth_setup.py`	autorização OAuth (1 vez)
`run_daily.py`	pipeline do dia
`batch.py`	produção em lote com agendamento
`post_now.sh`	ponto de entrada para o agendador
`agents.md`	contexto do projeto para o agente (economia de tokens)
`state.json`	temas já usados + histórico

---

10. Checklist para o primeiro vídeo no ar

- [ ] Canal do YouTube criado.
- [ ] YouTube Data API v3 ativada no Google Cloud.
- [ ] App OAuth publicado (não deixe em “Teste”).
- [ ] `credentials/client_secret.json` no lugar.
- [ ] `python3 auth_setup.py` executado com sucesso.
- [ ] `python3 upload.py` mostrando o canal correto.

- [ ] `python3 run_daily.py --dry-run` gerando um MP4 de 1080x1920 com áudio.
- [ ] `python3 run_daily.py --upload --privacy unlisted` para o primeiro teste real.
- [ ] Conferir no YouTube Studio, ajustar o que quiser em `config.py`.
- [ ] Trocar para `--privacy public` e criar a automação (Seção 5).